

PLAN DE GESTIÓN DE LA CALIDAD DEL SOFTWARE (PGCS) EN EMPRESA DE SERVICIOS

Control de Cambios del Documento

Versión	Fecha	Autor/es	Breve descripción del cambio
0.80	18-12-2019	MM	Creación
0.81	18-12-2019	MM, RR	Análisis Preliminar
0.90	26-12-2019	MM	Ampliación
0.91	26-12-2019	MM	Completamiento
0.99	26-12-2019	MM, ZZ	Revisión
1.00	06-01-2020	MM, ZZ	Presentación al cliente

Definiciones y conceptos asociados

Término	Definición / Conceptos
Aseguramiento de la Calidad (<i>Quality Assurance</i>)	QA: se enfoca en la <u>prevención de defectos</u> . Reconoce que las fallas en el proceso se pueden prevenir. El aseguramiento de la calidad garantiza que los enfoques, técnicas, métodos y procesos están diseñados para que los proyectos se implementen correctamente. Las actividades de QA monitorean y verifican que los procesos (Análisis, Diseño, Codificación, Pruebas y Evaluaciones, Gestión de la Configuración, etc.), utilizados para administrar y crear los entregables, se hayan seguido y estén operativos. QA es un proceso proactivo y es de prevención por naturaleza. QA <u>debe completarse antes</u> de QC.
Control de la Calidad (<i>Quality Control</i>)	QC: El control de calidad se centra en <u>identificar un defecto</u> . Reconoce que los defectos son casi inevitables y que se pueden reducir a un menor costo cuanto más anticipada sea su detección. Garantiza que los enfoques, técnicas, métodos y procesos que se diseñan en el proyecto se siguen correctamente. Las actividades de QC monitorean y verifican que los entregables del proyecto (iniciales, intermedios y finales) cumplan con los estándares de calidad definidos. El control de calidad es un proceso reactivo y es, esencialmente, de detección. QC <u>debe completarse después</u> de QA.

Full-Stack Developer	Es un desarrollador completo capaz de desarrollar software para el Back End y el Front End, así como conocer los componentes que hay entre ambos. Tiene capacidad para entender las necesidades del cliente y/o usuario y especificarlas, realizar el diseño UI, UX e interfaces con APIs, modelado de datos y tener conocimientos de arquitectura de servidores, redes y sistemas operativos. Es un desarrollador multiuso, capaz de ser el responsable del proyecto o de la parte que se le asigne, sea en relación con el Front End, el Back End, las APIs, la BD, como con actividades típicas de DevOps (integración, despliegue, automatización y monitoreo).
User Experience (UX)	La "experiencia del usuario" abarca todos los aspectos de la interacción del usuario final con la empresa, sus servicios y sus productos.
UX Design (UxD)	Es el proceso de diseño que utilizan los equipos para crear productos que brindan experiencias significativas y relevantes para los usuarios. Esto implica el diseño de todo el proceso de adquisición e integración del producto, incluidos los aspectos de marca, diseño, usabilidad y funcionalidad.
UX Interaction Design (UxI)	Es la técnica de diseño que busca facilitar las acciones que el usuario quiere realizar con un dispositivo a través de la interfaz de usuario, de manera de establecer las mejores relaciones entre el dispositivo que se usa, el usuario y la interfaz que se utiliza.
Jefe de Pruebas	Es el rol de quien tiene la responsabilidad general de establecer y gestionar una estrategia de pruebas basadas en el riesgo. El Jefe de Pruebas, con frecuencia, requerirá la participación del Analista de Pruebas Técnicas para garantizar la correcta implementación de un enfoque basado en riesgos.
Analista de Pruebas	El Analista de Pruebas es el encargado de medir y reportar los riesgos de producto, los defectos, las pruebas y la cobertura de una forma específica durante el proyecto o en operación. La confianza, aunque puede medirse a través de estudios, normalmente se reporta de forma subjetiva. La recopilación de la información necesaria para dar soporte a estas métricas forma parte del trabajo diario del Analista de Pruebas. Se espera que el Analista de Pruebas intervenga durante todo el proyecto y esté disponible para el equipo.
Analista de Pruebas Técnicas	Los Analistas de Pruebas Técnicas trabajan dentro del marco de trabajo de las pruebas basadas en riesgos, establecido para el proyecto por el Jefe de Pruebas, aportando su conocimiento de los riesgos técnicos que son inherentes al proyecto, tales como los riesgos asociados a la seguridad, la fiabilidad, la compatibilidad y el rendimiento del sistema.
Probador de Software	Las tareas típicas completas del probador de software incluyen, entre otras: revisar y contribuir a los planes de pruebas; analizar, revisar y evaluar los requisitos de usuario, las especificaciones y los modelos para su capacidad de ser probado; crear especificaciones de prueba; configurar el entorno de pruebas (a menudo en coordinación con los

	equipos de administración de sistemas y gestión de redes); preparar y obtener datos de prueba; implementar pruebas en todos los niveles de prueba, ejecutar y registrar las pruebas, evaluar los resultados y documentar las desviaciones de los resultados esperados; utilizar herramientas de administración o gestión y herramientas de seguimiento de prueba según proceda; automatizar pruebas (puede contar con el soporte de un desarrollador o un experto en automatización de pruebas); medir el rendimiento de los componentes y sistemas (si procede); y revisar pruebas desarrolladas por terceros. En general, los probadores de software a nivel de componente e integración son los desarrolladores, los probadores a nivel de aceptación funcional son los expertos del negocio y usuarios, y los probadores de aceptación operativas son los operadores (tecnología, seguridad, etc.).
Ingeniero de Automatización de Pruebas	Es el rol de quien tiene un amplio conocimiento de las pruebas, en general, y un profundo conocimiento de la teoría y la práctica de la automatización de pruebas, suficiente para influir en la dirección que toma una organización y/o proyecto a la hora de diseñar, desarrollar y mantener soluciones de automatización de pruebas para pruebas funcionales.
Partes interesadas (Stakeholders)	En un proyecto, las partes interesadas son cualquier individuo, grupo u organización que forme parte o se vea afectado por el mismo, obteniendo algún beneficio o perjuicio. Cada organización tiene sus partes interesadas, también conocidas como grupos de interés o públicos de interés.
Verificación y Validación Independiente (IV&V)	Los procesos de IV&V permiten proporcionar una evaluación objetiva de los productos y procesos de software durante todo su ciclo de vida en un entorno organizacional y operativo libre de la influencia, orientación y control del equipo de desarrollo. Cuando se aplica este enfoque dentro del propio equipo de desarrollo se recurre a IV&V "cruzadas" entre personas con diferentes roles (Ej: Desarrollador "A" evalúa los entregables del Desarrollador "B").
Calidad de Diseño	En el desarrollo de software, la calidad de concepción o de diseño comprende la calidad de los requisitos, de las especificaciones funcionales y no funcionales, y del diseño del sistema, propiamente dicho.
Calidad de Concordancia	La calidad de concordancia, de conformidad o técnica se refiere al nivel construcción de productos de software conforme a los requisitos, a las especificaciones técnicas y al diseño elaborado. La calidad de concordancia es un aspecto centrado principalmente en la implementación. Es el grado de correspondencia del producto implementado con los modelos previamente concebidos. Si la implementación (codificación y pruebas) sigue al diseño, y el sistema resultante cumple con los objetivos de requisitos funcionales y no funcionales, la calidad de concordancia es alta.

1 OBJETIVO

Describir cómo se gestionará la calidad durante todo el ciclo de vida del proceso de desarrollo de software, estableciendo las políticas, responsabilidades, normas de cumplimiento y actividades de calidad definidos por LA EMPRESA para todos los proyectos cuyos entregables estén constituidos por componentes, aplicaciones y sistemas de software desarrollados por la Gerencia de Operaciones y Tecnología.

2 ALCANCE

La gestión de la calidad de software incluye las actividades de aseguramiento de la calidad (*Quality Assurance*, enfocada en el proceso de desarrollo y mantenimiento del software) y de control de la calidad (*Quality Control*, sobre el producto de software en etapas tempranas o finales del proyecto), adaptadas a las metodologías ágiles de desarrollo de software, en general, e integradas a SCRUM, de manera particular. Además, la gestión de la calidad contempla un esfuerzo dedicado a las personas que participan o intervienen en el entorno del desarrollo y el uso del software, dirigido a mejorar su desempeño con habilidades en diferentes roles bajo un concepto de *Full-Stack Developer*, así como en la incorporación de principios, métodos y habilidades de UX (*User Experience*), UxD (*UX Design*) y UxI (*UX Interaction Design*) para diseñar la usabilidad del software en base a las necesidades, habilidades y limitaciones particulares del usuario de LA EMPRESA.

El presente plan de gestión de la calidad de software será el soporte fundamental de la calidad de los entregables de software que sean requeridos como resultado de los proyectos que sean formulados y administrados por LA EMPRESA. Sin embargo, deberá considerarse que este plan no sustituirá al Plan de Calidad que todo proyecto requiere para su ejecución, según las buenas prácticas recomendadas en la *GUÍA DE LOS FUNDAMENTOS PARA LA DIRECCIÓN DE PROYECTOS (GUÍA DEL PMBOK)*® del PMI (<https://www.pmi.org/pmbok-guide-standards/foundational/pmbok>).

Por otra parte, si bien estos objetivos, actividades, prácticas y procedimientos pueden ser de utilidad, este plan no contempla las necesidades particulares de calidad que se deberán requerir para el desarrollo de componentes, aplicaciones y sistemas de software que LA EMPRESA contrate a otros proveedores, los cuales deberán ser establecidos en el Plan de Calidad del proyecto de tercerización que se formule para cada caso.

3 MODELO DE CALIDAD DE SOFTWARE Y DE DATOS

La gestión de la calidad de software incluye los procesos de aseguramiento de la calidad (QA) y de control de la calidad (QC), con referencia al Sistema de Calidad definido por el estándar ISO 25000 (<https://iso25000.com/index.php/normas-iso-25000>), tanto para software como para datos, adaptados a las metodologías ágiles de desarrollo de software en general, y SCRUM, de manera particular.

Para el apoyar el proceso de desarrollo de software este plan adoptará el paradigma Ágil (<https://agilemanifesto.org/iso/es/manifesto.html>) y los principios, actividades, flujo de trabajo, roles y prácticas de SCRUM (<https://www.scrumguides.org/docs/scrumguide/v2017/2017-Scrum-Guide-Spanish-SouthAmerican.pdf>). En particular, para disponer de un lenguaje común, preparar roles y

desarrollar perfiles con habilidades sobre tipos, técnicas y herramientas de Pruebas de Software se guiará por el programa del ISTQB (*International Software Testing Qualifications Board*, <http://www.sstqb.es/recursos/descargas.html>).

4 METODOLOGÍA DE IMPLEMENTACIÓN Y EVALUACIÓN DE RESULTADOS

La implementación de las mejoras de calidad se llevará a cabo mediante una metodología evolutiva ejecutada en 5 fases. Cada fase durará 4 Sprints (40 días hábiles cada una). Las fases en las que se consideren objetivos de mejoras más difíciles de lograr se podrán desarrollar en 5 Sprints.

En cada fase se gestionarán las acciones del PGCS mediante un ciclo "Planificar-Hacer-Verificar-Actuar" o Círculo de Deming (<https://www.pdcahome.com/5202/ciclo-pdca/>), que acompañará el desarrollo de una aplicación o sistema de software que se esté gestionando con la metodología SCRUM. En "paralelo" todos los equipos participantes del PGCS desarrollarán actividades que permitan la incorporación gradual de mejoras de la calidad del software, considerando la evolución descrita en el apartado "8. FASES Y METAS PARCIALES". Estas mejoras se continuarán durante las actividades de mantenimiento de software, cualquiera sea el alcance del mismo.

La lógica de cada fase será la siguiente, con los ajustes evolutivos que se requieran:

SEMANA	FASE / PDCA	ACTIVIDADES PDCA APLICADAS AL PGCS	PROCESOS SCRUM
1	PLANIFICAR	Especificar Objetivos de Calidad	Sprint 1
		Identificar Roles Clave y Riesgos que podrían limitar / impedir Objetivos de Calidad para la Fase	
2		Motivar y Concientizar a equipos	
		Comunicar Acciones de Calidad y de Mitigación de Riesgos	
		Capacitar y entrenar a RRHH	
3	HACER	Capacitar y entrenar a RRHH	Sprint 2
		Implementar Calidad en el Proceso de Software (Verificaciones tempranas de Requisitos, de Diseño, de Código y de Pruebas) para maximizar fortalezas de SCRUM	
4		Capacitar y entrenar a RRHH	
		Implementar Métricas de Calidad de Proceso (Calidad de Especificación y Calidad de Concordancia) y de Producto (Calidad de Diseño y Calidad del Código)	
		Implementar Acciones de Gestión de Pruebas (Organización, Planificación, Estimación, Monitorización y Control de la Prueba)	
5		Capacitar y entrenar a RRHH	Sprint 3
		Implementar Controles de Calidad del Producto (Niveles, Tipos, Técnicas y Herramientas de Pruebas)	

SEMANA	FASE / PDCA	ACTIVIDADES PDCA APLICADAS AL PGCS	PROCESOS SCRUM
6		Capacitar y entrenar a RRHH	
		Implementar Acciones de Calidad en el Entorno del Software (Ambientes de Des/Pba/PreProd/Prod, Gestión de Configuración, Comunicaciones y Reportes)	
7	VERIFICAR	Auditar / Revisar niveles de calidad logradas en el proceso de desarrollo	Sprint 4
		Inspeccionar calidad (de muestras) de subproductos y productos de software	
		Verificar logros de Objetivos de Calidad	
8	ACTUAR	Identificar causas que limitaron o impidieron mejoras de Calidad	
		Adoptar Medidas Correctivas de Calidad	
		Adoptar Medidas Preventivas de Calidad	
		Difundir resultados y acciones de ajuste en base a Objetivos de Calidad	
		Mejoras específicas en la capacitación y entrenamiento de los RRHH	

En los proyectos que den lugar a entregables de software con desarrollo o mantenimiento por parte de LA EMPRESA, todas las partes interesadas internas deberán ser informados de las acciones de calidad que serán llevadas a cabo en cada fase del PGCS. El Jefe del Equipo de Calidad y los PMs involucrados deberán actuar coordinadamente con este propósito.

Esta metodología facilita evaluaciones parciales de resultados en cada ciclo, durante la actividad **Verificar** y permite realizar ajustes durante el momento **Actuar**, en función de las mediciones de atributos de calidad definidos y del resultado de las revisiones de las mejoras implementadas.

Durante, aproximadamente, 12 meses de evolución del plan, las fases sucesivas se enfocarán en consolidar los logros alcanzados y la resolución de conflictos de interés entre calidad y agilidad, o entre calidad y proyecto, con la finalidad de madurar un concepto moderno de productividad (entrega a tiempo con atributos de calidad preestablecidos y el "retrabajo" como excepción).

5 ORGANIZACIÓN Y RESPONSABILIDADES

La organización del Sistema de Calidad de Software contempla 3 equipos y las siguientes responsabilidades:

A 1. Equipo Gestión (PM + Scrum Master):

- El PM debe documentar las expectativas del cliente (externo) y los objetivos consensuados de gestión de calidad a nivel del proyecto que se recolecten y guíen ajustes particulares de cada proyecto en la implementación del PGCS.
- El PM debe garantizar que las partes interesadas, especialmente internas, respalden y acompañen las acciones de todos los objetivos de calidad incluidos en el PGCS en cada

fase. Cuando sea necesario, el PM mantendrá un Plan de Comunicaciones con la Entidad externa para transformarlo en un Stakeholder positivo.

- El Scrum Master (SM) debe determinar los procedimientos necesarios (ejemplos: reglas de especificación de requerimientos, cualquier revisión técnica independiente, inspecciones de código, revisiones de arquitectura del software, esfuerzo de pruebas independientes a cumplir por los desarrolladores, etc.) para alcanzar el nivel objetivo de calidad del proceso (QA) y de dosificación gradual del control de calidad (QC) en cada ambiente y por niveles de pruebas, de acuerdo a la experiencia de sus desarrolladores, fase del ciclo del vida del proyecto en que se encuentren y acorde a cada iteración o sprint particular.

A 2. Equipo Calidad (Área QA y roles designados en una fase particular; ejemplo: Seguridad, Tecnología)

- Realizar el máximo esfuerzo y compromiso con la entrega a tiempo de un producto que cumpla con los atributos de calidad definidos.
- Desempeñar un papel activo para garantizar que los objetivos de calidad del proyecto estén claramente articulados con las mejoras contempladas en cada fase del PGCS y que el Equipo Gestión comprenda los estándares, procedimientos y buenas prácticas profesionales esenciales que deben incorporarse al proyecto y al proceso.
- Mantener los compromisos para completar su parte del proyecto, sosteniendo la evolución de las mejoras contempladas en el PGCS.
- Monitorear la calidad de su propio trabajo.

A 3. Equipo Negocio (Entidades + Cliente externo)

- Comprender el papel del usuario y del cliente en el éxito del proyecto y del PGCS, garantizando una relación fluida para presentar requerimientos y cambios al software de manera gestionada, realizar los esfuerzos de control de calidad y pruebas en la oportunidad, intensidad y compromiso con la calidad requeridos por los equipos Gestión y Calidad.
- Trabajar con el cliente anticipadamente en el Alcance del Proyecto y en la Definición de Requisitos para determinar las necesidades del cliente, refinando esos requisitos con las restricciones de seguridad, cumplimiento de normas / estándares y otras exigencias operativas, considerando el costo / beneficio de todas las mejoras de calidad.
- Aplicar en cada despliegue de aplicaciones de software el Plan de Validación de Software LA EMPRESA para guiar las actividades de validación y aceptación del usuario interno y externo. Proponer mejoras para el Plan de Validación de Software de LA EMPRESA, en función de la experiencia de usuario.

6 POLÍTICAS DE CALIDAD DE SOFTWARE

6.1 ACTOR CLAVE: Scrum Master.

CALIDAD DE REQUERIMIENTOS: las especificaciones de requerimientos de software como historias de usuarios requieren la participación sostenida (todo el tiempo) del usuario clave del negocio, de expertos en seguridad y tecnologías, del representante de calidad y del scrum master, con la finalidad

de desarrollar la mejor comprensión de la funcionalidad, sus características funcionales, reglas de negocio asociadas, atributos no funcionales a satisfacer y criterios de aceptación, hasta agotar el criterio de calidad resumido en la regla "INVEST". La conversación particular sobre la comprensión detallada de la historia de usuario debe continuar durante el sprint cuando la historia sea asignada a un desarrollador y el mismo requiera precisiones y aclaraciones.

6.2 ACTOR CLAVE: Desarrollador, cualquiera sea su nivel de habilidades.

VALIDACIÓN TEMPRANA DE REQUISITOS Y PRUEBAS ASOCIADAS: la primera actividad del desarrollador al inicio del Sprint debe ser describir el o los casos de prueba derivados del escenario de uso, de sus pre y postcondiciones y de los criterios de aceptación enunciados en la historia de usuario asignada. Los casos de prueba del desarrollador deberán ser aprobados por el equipo calidad e integrados al plan de validación de software LA EMPRESA. El equipo calidad decidirá su automatización y la desarrollará con las herramientas apropiadas. Los casos de prueba en el ambiente de desarrollo deberán ser ejecutados bajo un enfoque de "Verificación y Validación Independiente - IV&V".

6.3 ACTOR CLAVE: Jefe del Equipo Calidad.

CALIDAD DE ARQUITECTURA DEL SOFTWARE: el equipo calidad promoverá la gestión del conocimiento enfocado en una arquitectura de componentes, aplicaciones y sistemas de software que garantice la seguridad en Internet, la fiabilidad en procesos críticos de negocios, la compatibilidad interna de interfaces propias y externa con componentes de terceros, y la mantenibilidad de los diferentes subproductos de software a nivel diseño y código, con atributos específicamente definidos en base a subcaracterísticas de calidad del estándar ISO 25000. La influencia y responsabilidad del equipo calidad será un factor clave de éxito para gestionar y extender el ciclo de vida de los entregables de software en el mediano plazo.

6.4 ACTOR CLAVE: Equipo Arquitectura de Aplicaciones o Arquitecto de Software.

CALIDAD DEL DISEÑO DE SOFTWARE: el ciclo de vida de componentes, aplicaciones y sistemas de software requiere diseño arquitectónico, revisiones de diseño y refactorizaciones del código que deben ser gestionadas por un equipo de idóneos en el software heredado (*legacy code*) o un experto en diseño, con la finalidad de mantener los atributos de calidad establecidos (seguridad, fiabilidad, compatibilidad y mantenibilidad). Esta política de calidad debe gestionarse de manera integrada con la enunciada en 6.3.

- 6.5 **ACTORES CLAVE:** Todos los perfiles que construyen, despliegan y operan software bajo una Metodología Ágil.

MÉTRICAS ÁGILES DE SOFTWARE: la medición del software en los equipos ágiles es una necesidad para guiar un proceso de mejora continua. Los desarrolladores de LA EMPRESA deberán modificar su cultura de programador individual hacia un enfoque de equipo ágil, incorporando habilidades para medir con transparencia, los procesos y productos del trabajo individual y del equipo. No hay mejora posible sin mediciones de software sostenidas en el tiempo.

7 ACCIONES DE GESTIÓN DE LA CALIDAD

- 7.1 **Adoptar la familia de normas ISO/IEC 25000 como estándar de calidad de software.**

Es necesario adoptar estándares de calidad para especificar atributos del software y de los datos en base a un modelo y a un lenguaje común.



<https://iso25000.com/index.php/normas-iso-25000>

- 7.2 **Aplicar el método de especificación de requerimientos de Scrum basado en historias de usuarios (H.U.) con sus características (funcionales y no funcionales) y los criterios de aceptación asociados. Utilizar las definiciones de los estándares ISO 25010 y 25012 para especificar atributos no funcionales. Verificar las historias de usuario y depurarlas antes de asignarlas para que satisfagan los criterios del Paradigma Ágil ("INVEST" para las H.U., "SMART" para las tareas asignadas en el Sprint).**

Un ejemplo de esquema simple para las historias de usuarios es el siguiente:

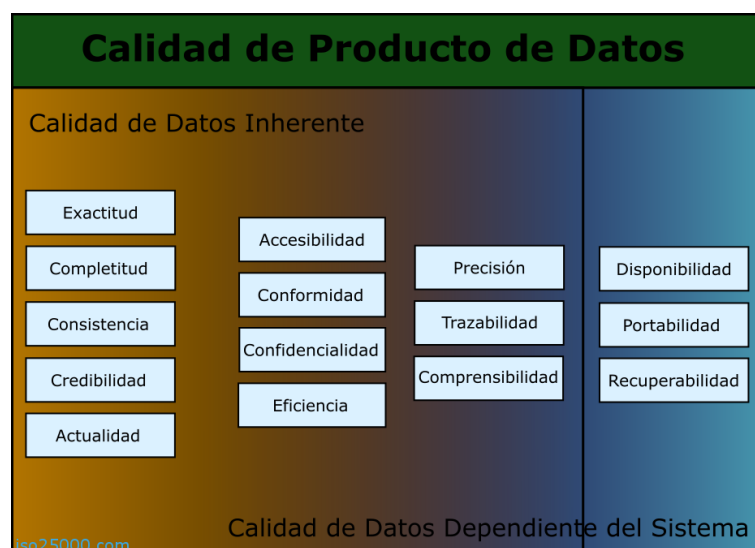
1 - HISTORIA DE USUARIO					2 - CRITERIO DE ACEPTACIÓN (CA)			
ID	ÉPICA	ROL	Funcionalidad / Característica	Razón/Resultado	ID CA	Escenario del CA / Precondiciones	Aclaración del Contexto	Resultado/Criterio de Aceptación
1	Necesito contar con una aplicación web donde se puedan ofrecer servicios y operaciones para tarjetas de débito.	Entidad	Quiero una operación que me permita transferir fondos de una tarjeta A a una tarjeta B de la misma entidad, efectuando PREVIAMENTE los siguientes controles y/o validaciones en el siguiente orden: 1.- Que la cuenta esté activa. 2.- Que la tarjeta esté activa. 3.- Que la tarjeta A (origen) cuente con los fondos suficientes. Luego de validados 1,2 y 3 quiero que se habiliten los códigos de movimientos correspondientes para identificar los mismos dentro del	Contar con los fondos transferidos de la tarjeta A en la Tarjeta B	1	1) Verificar el saldo anterior a la transferencia 2) Verificar que la cuenta origen A no arroje saldo negativo al transferir los fondos a la cuenta B 3) Verificar que, tanto la tarjeta origen como la destino, se	1) Si consulto saldo anterior y se le resta el importe de la transferencia debe darme como resultado el nuevo importe actualizado 2) No puede efectuarse transferencia con saldo negativo en cuenta origen	1) Si poseo un saldo anterior igual a 100 y realizo una transferencia de 75, debo finalizar la operación con un nuevo saldo actualizado de 25 2) Si el saldo anterior a la transferencia es 100 y realizo una transferencia por 120, la misma debe ser rechazada 3) Si la transferencia se realiza sobre una tarjeta origen o destino con un estado diferente a Normal, la operación debe finalizar con

Las características o atributos de calidad de la ISO 25010 para requerimientos no funcionales del software pueden ser:



<https://iso25000.com/index.php/normas-iso-25000/iso-25010>

Por similitud, para características de calidad de las bases de datos, se empleará:



<https://iso25000.com/index.php/normas-iso-25000/iso-25012>

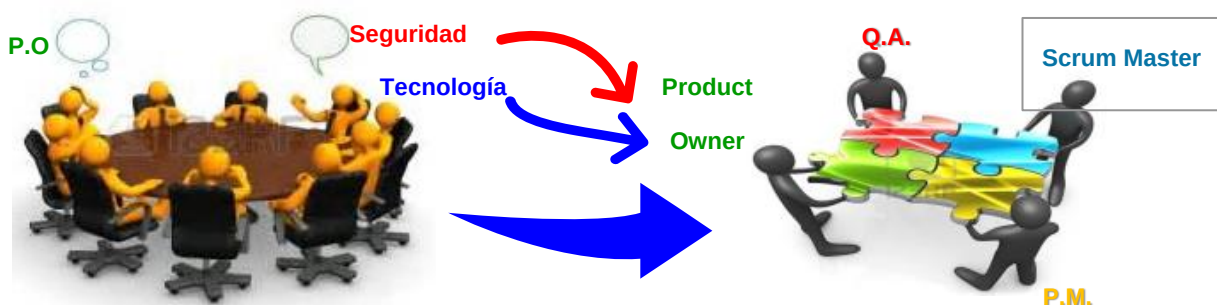
Antes de asignar una H.U a un desarrollador, se deben verificar las características resumidas en la regla nemotécnica "INVEST" y las tareas para su diseño, construcción y pruebas deberán verificar la

regla "SMART". Estas revisiones constituyen actividades de aseguramiento de la calidad que contribuyen a las entregas con valor, a tiempo y un equilibrio adecuado de compromiso y calidad:

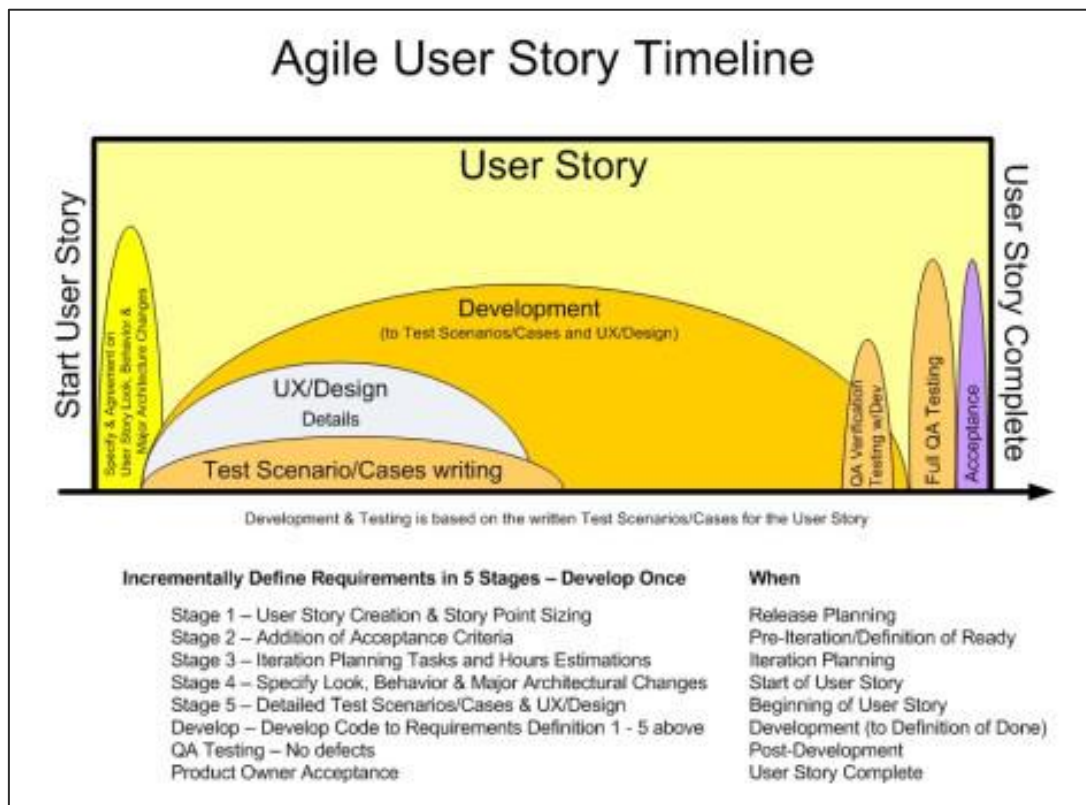


<https://xp123.com/articles/invest-in-good-stories-and-smart-tasks/>

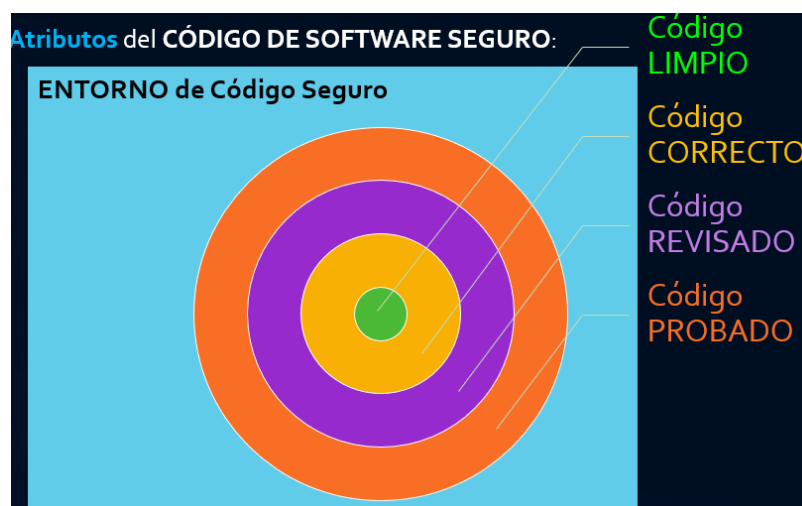
7.3 Bajo la coordinación del Scrum Master, incorporar a partir del momento del Sprint Planning a los especialistas de operaciones (Seguridad, Tecnología, etc.) que junto al Product Owner, al Project Manager y al líder o representante de QA, establecerán los criterios de aceptación "no funcionales" (seguridad, fiabilidad, portabilidad, etc.) y las necesidades para su construcción y pruebas. Durante el Sprint, mantener las conversaciones de detalle que sean necesarias para cada desarrollador, de manera de que las características no funcionales del software y de los datos sean explicadas con más detalles o ejemplos que eliminen las ambigüedades de diferentes interpretaciones y se ajusten con las expectativas de las partes interesadas representadas por el Project Manager.



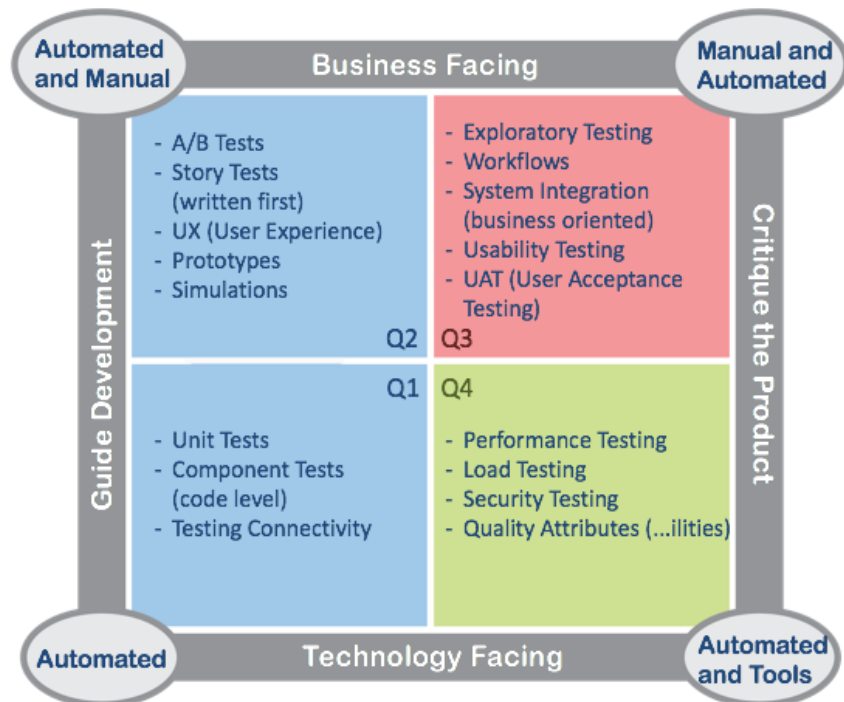
7.4 Desarrollar habilidades homogéneas en todos los integrantes de los equipos gestión y negocio para especificar historias de usuarios con sus criterios de aceptación y especificar / diseñar casos de prueba, construir, probar, validar y aceptar las funcionalidades para una versión determinada de un sistema de software, en una secuencia típica de un enfoque ágil.



7.5 Desarrollar habilidades individuales en todos los desarrolladores para implementar (codificar y probar) código limpio, código correcto, código revisado y código probado, en un entorno de Código Seguro.

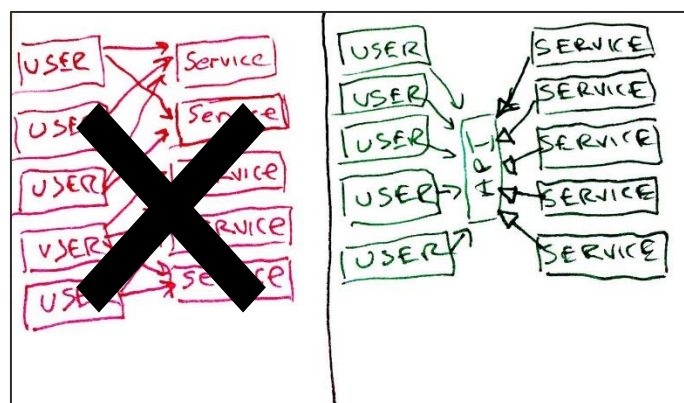


7.6 Desarrollar habilidades individuales en todos los desarrolladores para hacer “*testing* cruzado” de unidades, componentes e interfaces de software (codificar y probar) con las técnicas de los cuadrantes Q1 y Q2 de la Matriz de Testing Ágil.



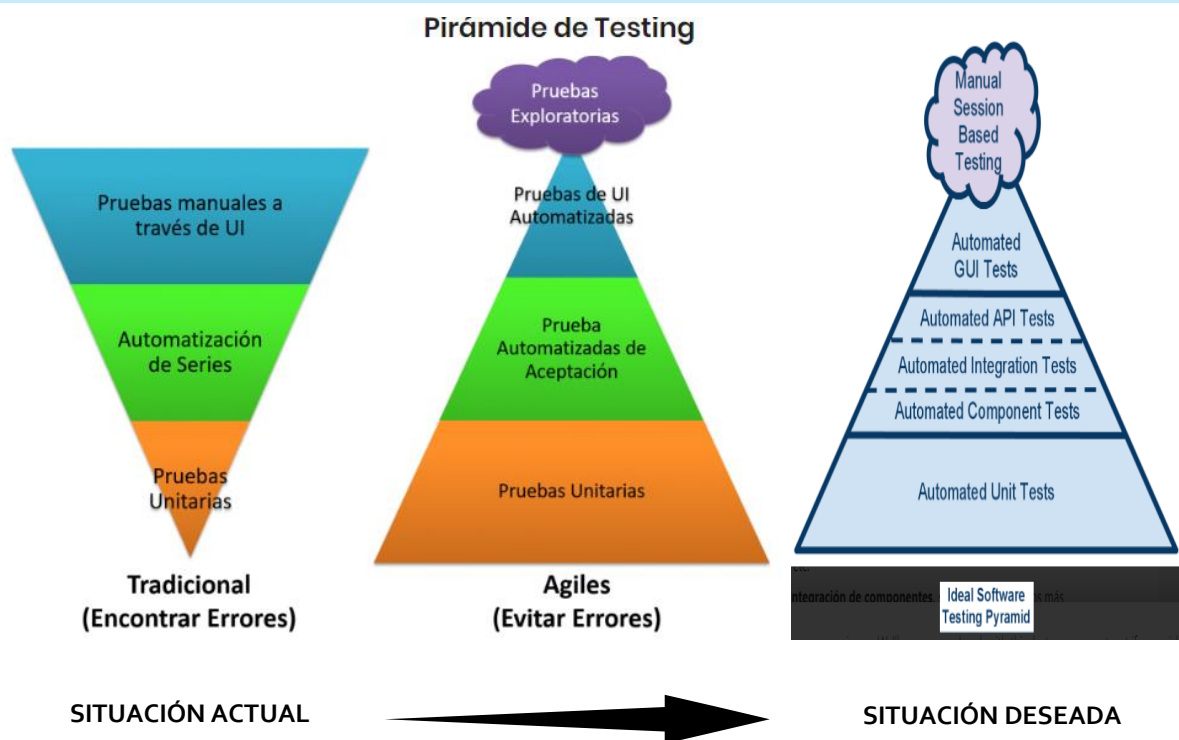
<https://www.scaledagileframework.com/agile-testing/>

7.7 Desarrollar habilidades de diseño de software en equipo en todos los desarrolladores en relación con la estructura, el funcionamiento y la interacción entre las partes del software, aplicando las mejores prácticas de calidad de concordancia y organizando el sistema mediante un enfoque de arquitectura de software seguro, fiable, mantenible y con desempeño eficiente.



7.8 Especializar y aplicar el trabajo del equipo Calidad en las pruebas de integración, regresión, aceptación, negativas, rendimiento, carga y seguridad. Se desarrollarán 5 perfiles, en base al enfoque del ISTQB: 1. Jefe del Equipo de Calidad (aseguramiento de la calidad), 2. Jefe de Pruebas (control de calidad), 3. Analista de Pruebas / Probador de Software, 4. Analista de Pruebas Técnicas y 5. Ingeniero de Automatización de Pruebas. El Jefe de

Pruebas tendrá responsabilidad de dosificar, en diferentes niveles de pruebas (de componentes, de integración, de sistema y de aceptación), los esfuerzos de pruebas para todos los cuadrantes, tanto para el equipo de Desarrollo como para el de Calidad. El trabajo de los especialistas será priorizado sobre los tipos y técnicas de pruebas de los cuadrantes Q3 y Q4 de la Matriz de Testing Ágil y serán los capacitadores de todos los desarrolladores. Esta acción de mejora deberá evolucionar las siguientes etapas para alcanzar el máximo posible con soporte de herramientas automatizadas:



La evolución debe evitar volver a invertir la Pirámide Ágil o Ideal (no volver al “cono de helado”)

7.9 Aplicar métricas de calidad en el trabajo de los equipos Calidad y Gestión para reportar información de gestión, medir efectividad y evaluar la eficiencia del trabajo. Se adoptarán las siguientes métricas: Gráficos de *Burndown*, Porcentaje de Ejecución de Caso de Prueba, Tasa de Aprobación de Caso de Prueba, Categoría de Defecto, Densidad de Defectos, Porcentaje de Detección de Defectos (DDP), Tiempo Medio de Detección (MTTD) y Tiempo Medio de Reparación (MTTR), entre otras.

7.10 Maximizar el uso y los beneficios de las herramientas de soporte al proceso (TFS, entre otras), incorporando otras que faciliten la automatización para guiar el trabajo con agilidad, gestionar el producto de software y reportar a tiempo información para la toma de decisiones.

8 FASES Y METAS PARCIALES

Fase 1 – Mejoras Elementales. Comprende las siguientes acciones:

- 7.1 Adoptar la familia de normas ISO/IEC 25000 como estándar de calidad de software.
- 7.2 Aplicar el método de especificación de requerimientos de Scrum basado en historias de usuarios con sus características (funcionales y no funcionales), los criterios de aceptación asociados. Utilizar las definiciones de las ISO 25010 y 25012 para especificar atributos no funcionales. Verificar las historias de usuario y depurarlas antes de asignarlas para que satisfagan los criterios del Paradigma Ágil (“INVEST” para las H.U., “SMART” para las tareas asignadas en el Sprint).
- 7.9 Aplicar métricas de calidad en el trabajo de los equipos Calidad y Gestión para reportar información de gestión y medir efectividad y eficiencia. Se adoptarán las siguientes métricas: Gráficos de *Burndown*, Porcentaje de Ejecución de Caso de Prueba, Tasa de Aprobación de Caso de Prueba, Categoría de Defecto, Densidad de Defectos, Porcentaje de Detección de Defectos (DDP), Tiempo Medio de Detección (MTTD) y Tiempo Medio de Reparación (MTTR), entre otras.
- 7.10 Maximizar el uso y los beneficios de las herramientas de soporte al proceso (TFS, entre otras) incorporando otras que faciliten la automatización para guiar el trabajo con agilidad, gestionar el producto de software y reportar a tiempo información para la toma de decisiones.

Las acciones 7.9 y 7.10 se implementarán gradualmente, evolucionando las mejoras durante las 5 fases.

A partir de la Fase 1, cada fase deberá ajustarse en función de los resultados medidos en la fase previa.

Fase 2 – Mejoras en la INGENIERÍA DE REQUERIMIENTOS. Comprende:

- 7.3 Bajo la coordinación del Scrum Master, incorporar a partir del momento del Sprint Planning a los especialistas de operaciones (Seguridad, Tecnología, etc.) que junto al Product Owner, al Project Manager y al líder o representante de QA, establecerán los criterios de aceptación “no funcionales” (seguridad, fiabilidad, portabilidad, etc) y las necesidades para la construcción y pruebas. Durante el sprint, mantener las conversaciones de detalle que sean necesarias para cada desarrollador, de manera de que las características no funcionales del software y de los datos sean explicadas con más detalles o ejemplos que eliminen las ambigüedades de diferentes interpretaciones y se ajusten con las expectativas de las partes interesadas representadas por el Project Manager.
- 7.4 Desarrollar habilidades homogéneas en todos los integrantes de los equipos Gestión y Negocio para especificar historias de usuarios con sus criterios de aceptación y especificar / diseñar casos de prueba, construir, probar, validar y aceptar las funcionalidades de una versión determinada de un sistema de software, en una secuencia típica de un enfoque ágil.
- 7.9 Aplicar métricas de calidad ...
- 7.10 Maximizar el uso y los beneficios de las herramientas ...

Fase 3 – Mejoras en CONTROL DE CALIDAD. Incluye:

- 7.5 Desarrollar habilidades individuales en todos los desarrolladores para implementar (codificar y probar) código limpio, código correcto, código revisado y código probado, en un entorno de Código Seguro.
- 7.6 Desarrollar habilidades individuales en todos los desarrolladores para hacer “testing cruzado” de unidades, componentes e interfaces de software (codificar y probar) con las técnicas de los cuadrantes Q1 y Q2 de la Matriz de Testing Ágil.
- 7.9 Aplicar métricas de calidad ...
- 7.10 Maximizar el uso y los beneficios de las herramientas ...

Fase 4 – MEJORAS EN ASEGURAMIENTO Y CONTROL DE LA CALIDAD. Tendrá en su alcance:

- 7.7 Desarrollar habilidades de diseño de software en equipo en todos los desarrolladores en relación con la estructura, el funcionamiento y la interacción entre las partes del software, aplicando las mejores prácticas de calidad de concordancia y organizando el sistema mediante un enfoque de arquitectura de software seguro, fiable, mantenible y con desempeño eficiente.
- 7.8 Especializar y aplicar el trabajo del equipo Calidad en las pruebas de integración, regresión, aceptación, negativas, rendimiento, carga y seguridad. Se desarrollarán 5 perfiles, en base al enfoque del ISTQB: 1. Jefe del Equipo Calidad (calidad de proceso), 2. Jefe de Pruebas (control de calidad), 3. Analista de Pruebas / Probador de Software, 4. Analista de Pruebas Técnicas y 5. Ingeniero de Automatización de Pruebas. El Jefe de Pruebas tendrá responsabilidad de dosificar en diferentes niveles de pruebas (de componentes, de integración, de sistema y de aceptación) los esfuerzos de pruebas para todos los cuadrantes, tanto para el equipo de Desarrollo como para el de Calidad. El trabajo de los especialistas será priorizado sobre los tipos y técnicas de pruebas de los cuadrantes Q3 y Q4 de la Matriz de Testing Ágil y serán los capacitadores de todos los desarrolladores. Esta acción de mejora deberá evolucionar gradualmente, incrementando la automatización de los procesos de software.
- 7.9 Aplicar métricas de calidad ...
- 7.10 Maximizar el uso y los beneficios de las herramientas ...

Fase 5 – OPTIMIZACIÓN DE MEJORAS. Comprenderá las acciones pendientes de mejores resultados que los realmente alcanzados y la profundización de las mejoras contempladas en las acciones 7.7, 7.8, 7.9 y 7.10.

- Acciones de mejoras que sean necesarias para cumplir las políticas de calidad definidas.
- Ampliación de las habilidades de diseño de software en equipo, en relación con 7.7.
- Mejoramiento del conocimiento y habilidades del equipo de calidad, en relación con 7.8.
- Completamiento de las métricas de calidad en función de los objetivos planteados en 7.9.
- Optimización del uso y de los beneficios de las herramientas de automatización y soporte al desarrollo, en relación con 7.10.